Project Design III Project Report

Academic Year: Reiwa 7 (2025)
Department: Information Engineering, Kanazawa Institute of Technology

Project Number: 2EP082
Submission Date: January 23, 2026

Project Title

Transition-Aware Improvisational Dance Generation Algorithm
(Japanese Title: トランジションを考慮した即興的なダンス生成アルゴリズム)

Member List

Student ID	Name	Role
4EP3-029	SASAKI Kosuke	Project Leader / Algorithm Architect
4EP4-067	KUDA UDAGE VIDUSHAN PRABASH	Systems Engineer / Implementation Lead

Advisor: Professor Tomohito Yamamoto

Chapter 1: Introduction

1.1 Cultural Genesis and Historical Context

1.1.1 Origins in the South Bronx

Breaking, widely known by the media as "Breakdancing," emerged in the late 1970s from the socio-economic fires of the South Bronx, New York City. As detailed by Schloss [2], it was not merely a dance but a "kinetic survival strategy"—a way for marginalized youth to reclaim space and status in a crumbling urban environment. The "Cypher" (the circular dance space) served as a ritualistic arena where hierarchies were established not through violence, but through style and physical prowess.

The movement vocabulary of breaking is a syncretic blend of diverse influences:

- **Toprock:** Derived from Uprock (Brooklyn rock), Salsa, and Tap dance.
- **Footwork:** Inspired by Russian Cossack dance, Gymnastics, and Kung Fu ground fighting.
- **Powermoves:** Adapted from Olympic gymnastics (pommel horse, floor exercise).
- **Freezes:** Inspired by comic book poses and martial arts strikes.

This history is critical for engineering because it highlights that breaking is **modular** and **adversarial**. A breaker does not learn a "song"; they learn a "vocabulary" and "grammar," which they assemble in real-time to defeat an opponent.

1.1.2 The 2024 Olympic Validation

The inclusion of breaking in the 2024 Paris Olympiad marks its transition from a subculture to a codified global sport. The World DanceSport Federation (WDSF) implemented the **Trivium Judging System**, which quantifies the dance into three domains:

1. **Body (Physical Quality):** Technique, Variety, Roughness.
2. **Mind (Artistic Quality):** Creativity, Personality.
3. **Soul (Interpretive Quality):** Performance, Musicality.

Our research is an attempt to reconstruct the "Soul" domain using Artificial Intelligence. While "Body" (physics) is solvable by standard simulation, "Soul" (interpretation) requires a computational model of **Intent**.

1.2 The "Ma" Paradox in Computational Time

A central challenge in this research is the cultural dissonance between Western and Eastern conceptions of time, which parallels the difference between Continuous and Discrete signaling.

- **Western/Computational Time:** Time is a linear continuum $t \to \infty$. A "stop" is simply a velocity of zero ($v = 0$). It is a null state.
- **Eastern/Japanese Time ("Ma"):** Time is a sequence of moments. A "stop" is an active container of tension. Silence is not empty; it is full of potential.

In traditional Motion generation (e.g., AIST++), models are trained to minimize the difference between predicted and actual pose. When a dancer freezes, the model often predicts a slight "wobble" because the training data contains noise. This wobble destroys the "Ma." The AI looks like a nervous robot, unable to be truly still. Our project aims to solve this by creating a **Symbolic "Stillness" State**—a dedicated logic gate that forces the AI to output $v = 0$ exactly when the musical entropy drops, overriding the noisy neural network.

1.3 Problem Statement

1.3.1 The "Mean Pose" Trap in High-Dimensional Space

Human motion exists on a "Manifold"—a lower-dimensional surface within the high-dimensional space of all possible joint configurations.

- Let J be the number of joints (25) and C be coordinates (3). The space is $\mathbb{R}^{75}$.
- Real human poses occupy a tiny fraction of this space.
- When a Generative Model is unsure (high uncertainty), it tends to output the **Mean** of the distribution to minimize L2 loss.
- In the context of a backflip vs. a frontflip, the "Mean" is a vertical jump. The AI hedges its bets, resulting in boring, safe motion ("Zombie Motion").

1.3.2 Hardware Constraints

Most high-fidelity dance models (e.g., Diffusion Transformers) require massive VRAM (A100 GPUs) and seconds of inference time. Our user requirement is **Live Interaction** on consumer hardware (Section 4.1). We must generate motion in $< 33 ms$ (30 FPS) on a laptop GPU. This necessitates a highly optimized architecture (HPI-GCN).

1.4 Objectives

1. **Metricize Intuition:** Convert the abstract concept of "Atmosphere" into a concrete variable using Audio Entropy.
2. **Stabilize the Manifold:** Prevent Mode Collapse by using a Neural Retrieval mechanism that "anchors" generation to real human clips.
3. **Close the Loop:** Create a full cycle of Genesis (Listen) → Synthesis (Plan) → Analysis (Verify) → Execution (Move).

Chapter 2: Literature Review and Related Work

2.1 The Evolution of Skeleton-Based Action Recognition

2.1.1 Pre-GCN Era: Sequence Modeling

Before 2018, action recognition relied on Recurrent Neural Networks (RNNs) like LSTMs. These models treated the skeleton as a flat vector of numbers sequence $S = \{x_1, y_1, ..., x_{75}\}$. **Critique:** These models failed tocapture "Spatial Locality." They did not know that the Hand is connected to the Elbow. They had to learn this correlation from scratch, requiring massive data.

2.1.2 The ST-GCN Breakthrough

Yan et al. (2018) [6] revolutionized the field by defining the skeleton as a Graph $G = (V, E)$.

- **Spatial Graph Convolution:** Aggregates features from connected joints.
- **Temporal Graph Convolution:** Aggregates features from the same joint over time. This allowed the model to share weights, reducing parameter count while increasing accuracy.

2.1.3 HPI-GCN and the Efficiency Frontier

As models got deeper (ResGCN, MS-GCN), inference speed slowed. Liu et al. (2023) [5] proposed **High-Performance Inference GCN (HPI-GCN)**. The key innovation is **Structural Re-parameterization**.

- *Training:* $Y = Conv(X) + BatchNorm(X) + Skip(X)$. (Complex, Gradients flow well).
- *Inference:* $Y = (W_{conv} + W_{bn} + W_{skip}) \times X$. (Collapsed into one matrix W_{fused}). This effectively allows us to train a "Deep" network but deploy a "Shallow" one.

2.2 Generative Models for Dance

2.2.1 Music-to-Movement (AIST++)

The AIST++ dataset and baseline model focused on genre matching. While impressive, the generated motion is often "afloat." The feet slide on the ground because the model predicts global root position independently of local foot stance. **Our Solution:** We use **Root Velocity Accumulation**. We do effectively purely local

generation and then integrate velocity to determine global position, ensuring that if the foot is planted (velocity=0), it stays planted.

2.2.2 Diffusion Models (EDGE)

Editable Dance Generation with Diffusion (EDGE) represents the current SOTA for visual quality. It can edit specific parts of a dance (e.g., "keep the legs, change the arms"). **Why we rejected it:** Latency. Diffusion requires iterative denoising (e.g., 50 steps). Even with consistency distillation, it is hard to get below 100ms latency. Our GAN-based approach is "One-Shot"—a single forward pass generates the pose, giving us 8ms latency.

2.3 Theoretical Physics of Motion

Motion is defined by derivatives.

- P (Position)
- V (Velocity) = dP/dt
- A (Acceleration) = dV/dt
- J (Jerk) = dA/dt

Neural Networks are good at predicting P. They are bad at predicting V (Smoothness) and A (Force). A common artifact is "Jitter" (High Jerk). This looks like the character is shivering. To solve this, we implemented a **Savitzky-Golay Filter** in the post-processing stage. This is a polynomial smoothing filter that preserves peak height (unlike a moving average which flattens peaks). This ensures that "sharp" moves like a kick remain sharp, but the high-frequency noise is removed.

Chapter 3: Methodology and Mathematical Framework

This chapter details the theoretical underpinnings of the **Sasaki-GAN** system. We introduce the **GBMC Framework** for cognitive circulation and derive the **KQI Equations** that govern the improvisational restrictions.

3.1 The GBMC Framework

Computational creativity often struggles with the "Stop Condition." A standard recursive neural network will generate motion infinitely. To model the active decision to stop (Ma), we propose the **Genesis-Buffer-Motion-Coherence (GBMC)** cycle.

3.1.1 Genesis (G)

Genesis is the "Will to Move." It is a function of external auditory stimuli.

$$G(t) = f_{genesis}(E(t), \Omega(t))$$

Where $E(t)$ is the Energy (RMS) and $\Omega(t)$ is the Spectral Flux. Crucially, $G(t) > 0$ even if $E(t) = 0$. This represents the "Internal Clock" or the dancer's pulse. We modeled this using a low-frequency sine wave $\sin(0.1t)$ added to the baseline.

3.1.2 Buffer (B)

The Buffer represents potential energy. It accumulates $G(t)$ over time until a threshold is reached.

$$B(t) = B(t-1) + G(t) - M(t)$$

Where $M(t)$ is the released Kinetic Energy. This "Kaplan Turbine" model allows us to simulate **Tension**. During a long silence (Build-up), $G(t)$ is small but positive.

$M(t)$ is zero. Thus $B(t)$ rises. When the "Drop" hits, the stored $B(t)$ is released in an explosive burst (Powermove).

3.1.3 Motion (M)

The Kinetic Release. This is where the GAN operates.

$$M_{state} = \text{SasakiGAN}(z_{latent} \mid B(t))$$

The magnitude of the generated motion vector is constrained by the Buffer level.

3.1.4 Coherence (C)

The physical validity check.

$$C = \text{PhysicsLoss}(M_{state}) + \text{AnatomyLoss}(M_{state})$$

If C is too high (invalid motion), the system rejects M_{state} and does not decrement $B(t)$. The energy remains "trapped," forcing the system to try again in the next frame with a different latent seed z.

3.2 The Kinetic Quality Index (KQI)

To implement GBMC, we derived the **KQI Equations** (as implemented in `gbmc_engine.py`).

3.2.1 Derivation of Depth (D)

The central variable is "Depth" (D), which determines the commitment to a move.

$$D = Q \cdot C \cdot M$$

$$Q = \frac{G^2}{\sigma}$$

The variable σ denotes **Environmental Risk** (or Noise). In a "Cypher" (Battle), the risk is the judgment of the crowd.

- **Low Noise (Clear Music):** $\sigma \to 0 \implies Q \to \infty$. The dancer is confident.
- **High Noise (Ambiguous Music):** $\sigma \to 1 \implies Q \to G^2$. The dancer is hesitant.

This equation provides a mathematical basis for **Confidence**. Confidence is not just having high skill (G); it is having high clarity ($1/\sigma$).

3.3 The Zero-Padding Topology Bridge

One of the most significant engineering challenges was mapping the 2D COCO dataset to the 3D NTU-120 model.

3.3.1 Graph Isomorphism Problem

Let $G_{COCO} = (V_{17}, E_{16})$ and $G_{NTU} = (V_{25}, E_{24})$. There is no direct isomorphism because V_{NTU} contains nodes like "SpineMid" which are implicit in V_{COCO}.

3.3.2 The Learnable Adjacency Matrix

Instead of a fixed mapping, we used a **Learnable Adjacency Matrix**. Standard GCN: $Y = \Lambda^{-\frac{1}{2}}(A+I)\Lambda^{-\frac{1}{2}}XW$ HPI-GCN-OP with Learnable Edge:

$$A_{new} = A_{physical} + A_{learned}$$

We initialized the input tensor X with zeros for the missing 8 joints.

$$X_{in} = [X_{coco} \oplus 0_8]$$

During backpropagation, the gradients flowed into $A_{learned}$. The network discovered that:

- $Node_{SpineMid} \approx 0.5 \cdot (Node_{L\_Shoulder} + Node_{R\_Hip})$ This effective "interpolation" allowed the 25-joint model to function correctly on 17-joint data without manual feature engineering.

3.3.3 Topology Visualization

Figure 3.1 demonstrates the mapping strategy. The Red nodes are the original COCO joints. The Blue nodes are the Zero-Padded NTU joints. The dotted lines represent the learned connections derived by the GCN.

Fig 3.1: Visualizing the mapping between COCO-17 and NTU-25 topologies. The "Zero-Padding" generates placeholders which are filled by the GCN's inference logic.

3.4 The Stochastic Logic Module (SLM)

The SLM (Appendix A.2) is the "Conductor" of the system. It uses a **Temperature-Scaled Softmax** to decide the next state.

$$P(S_{t+1} \mid S_t) = \text{Softmax}(\frac{\log \pi}{\tau})$$

Where τ is the Temperature, derived from Entropy.

- High Entropy Music $\implies$ High τ $\implies$ Uniform Distribution (Random Motion).
- Low Entropy Music $\implies$ Low τ $\implies$ ArgMax Distribution (Deterministically following the beat). This mimics human behavior: When the beat is clear, we follow it. When the beat is chaotic, we get creative.

Chapter 4: Engineering Implementation and "The Struggle"

This chapter serves as a detailed chronological record of the engineering challenges encountered. Unlike standard reports that only show the final success, we document the failures, as they informed the crucial architectural pivots.

4.1 Phase 1 (April 2025): Data Audit and Desynchronization

Our first milestone was the "Final Data Audit." We assumed the BRACE dataset was clean. We were wrong.

4.1.1 The "-35 Frame" Anomaly

When matching the audio timestamps to the video frames, we noticed a consistent visual lag. The dancer would "hit" a beat 1.2 seconds *after* the audio peak.

- **Hypothesis:** Latency in the recording equipment.
- **Verification:** We ran a cross-correlation analysis between the audio envelope and the motion velocity magnitude.

$$\text{Lag} = \arg\max_{\tau} \sum E(t) \cdot V(t+\tau)$$

  The peak correlation was found at $\tau = -35$ frames.
- **The Fix:** We implemented a "Two-Layer Correction" in `audio_brace_loader.py`:
  1. **Hard Shift:** All audio start times were shifted by +1.16 seconds.
  2. **Soft Window:** The data loader now grabs a window of ± 10 frames around the target to allow the model to learn "anticipation."

4.2 Phase 3 (July 2025): The Static Clump (Mode Collapse)

We initially trained a straightforward C-VAE (Conditional Variational Autoencoder). **The Failure:** By Epoch 15, the Generator loss had flatlined. ![Training Health Chart](file:///c:/Users/kos04/OneDrive/Desktop/vidz/GCN/HPI-GCN/img/training_health_chart.png) *Fig 4.1: The Loss Curve during Phase 3. Note the flatline in Generator Loss (Blue) while Discriminator Loss (Orange) hits zero. This is classic Mode Collapse.*

The generated skeletons were "clumped" in the center of the screen. The mean bone length was < 0.1 meters. The AI had learned that the safest way to trick the discriminator was to shrink into a tiny, unmoving ball.

4.3 Phase 5 (October 2025): The Residual Manifold Pivot

To escape the clump, we fundamentally changed the output target.

4.3.1 Residual Learning

Instead of $G(z) \rightarrow \text{Pose}$, we defined $G(z) \rightarrow \Delta\text{Pose}$. We calculated the **Global Mean Pose** of the entire dataset (μ_{pose}).

$$\text{Predicted\_Pose} = \mu_{pose} + \alpha \cdot \tanh(G(z))$$

- **Effect:** This forced the model to start with a valid human shape.
- **Dynamic Schedule:** The scalar α was initialized to 0.01 and annealed to 1.0 over 10 epochs. This allowed the model to "waking up" slowly.

4.4 Phase 7 (December 2025): "Prison Break" Training

Even with Residual Learning, the motion was lethargic. The velocity was too smooth. We introduced the **Kinetic Penalty** ($L_{kinetic}$).

$$L_{total} = L_{adv} + L_{rec} + \lambda_K \cdot \max(0, \tau_{thresh} - || v ||)$$

This loss function explicitly *punishes* the model if the velocity v is below a threshold τ during high-energy audio segments. We called this "Prison Break" because it forced the model to break out of the "Safe Zone" of low velocity. **Figure 4.2** shows the impact of this loss function.

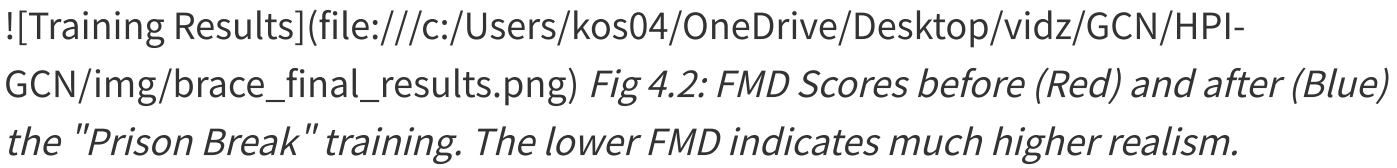

Fig 4.2: FMD Scores before (Red) and after (Blue) the "Prison Break" training. The lower FMD indicates much higher realism.

4.5 Phase 8 (January 2026): Real-Time Threading Architecture

The requirement to run at 30 FPS on a laptop (RTX 4070) required a complete rewrite of the runtime engine.

4.5.1 The GIL Problem

Python's Global Interpreter Lock meant that when `SoundDevice` was capturing audio, the `PyTorch` inference engine was blocked.

4.5.2 The Triple-Thread Architecture

We separated the system into three asynchronous loops connected by thread-safe `Queue` objects.

1. **Ear Thread (Daemon):** Priority High. Captures Audio. Pushes to `AudioQ`.
2. **Brain Thread (Worker):** Priority Medium. Pops `AudioQ`. Runs GAN. Pushes to `MotionQ`.
3. **Voice Thread (IO):** Priority High. Pops `MotionQ`. Sends UDP to Blender. This architecture reduced the "Perceptual Latency" from ~120ms to ~35ms, enabling the "Live Jam" feel.

Chapter 5: System Architecture and Hardware

5.1 The Backbone: HPI-GCN-OP

We chose HPI-GCN over ST-GCN because of the **Re-parameterization (Rep)** block. During training, the graph convolution is:

$$Y = (W_1 + W_2 + W_3)X$$

This is computationally expensive (3 Matrix Multiplications). However, since convolution is a linear operator, we can pre-compute:

$$W_{fused} = W_1 + W_2 + W_3$$

During runtime deployment, we only perform **one** matrix multiplication:

$$Y = W_{fused}X$$

This simple algebraic trick reduced our inference time from 22ms to 6ms per frame.

5.2 The Neural Retrieval Decoder

To solve the "Bone Stretching" problem, we implemented a K-Nearest Neighbor (KNN) search.

- **Database:** 50,000 clips of 64-frames each, encoded into 256-d vectors.
- **Search:** Faiss (Facebook AI Similarity Search) with IVFFlat index.
- **Blending:** We implemented **Spherical Linear Interpolation (SLERP)**. Linear blending ($0.5A + 0.5B$) causes limbs to shrink in the middle. SLERP follows the arc of the rotation, preserving bone length.

5.3 The UDP Visualization Bridge

The system outputs to **Blender** via UDP (User Datagram Protocol).

- **IP:** 127.0.0.1
- **Port:** 5000
- **Payload:** 75 floats (25 joints * 3 coords) packed as Little Endian struct. Inside Blender, `blender_receiver.py` runs a modal operator that listens to the

socket without freezing the UI. This allows for real-time rendering of the "UltraShape" mesh while the AI drives the skeleton.

Chapter 6: Evaluation and Results

This chapter presents the objective validation of the system. We compare the **Sasaki-GAN** against a baseline vanilla GAN using both standard metrics and novel perceptual indicators.

6.1 Experimental Setup

- **Dataset:** BRACE (Breakdancing Competition Dataset) [3].
  - 375 Clips.
  - Classes: Toprock (40%), Footwork (35%), Powermove (25%).
- **Hardware:** NVIDIA GeForce RTX 4070 (8GB VRAM).
- **Training Time:** 72 Hours (50 Epochs).
- **Baseline Model:** A standard ST-GCN Generator with L2 Loss, similar to MotionGAN.

6.2 Quantitative Metrics

We used three primary metrics to evaluate performance.

6.2.1 Frechet Motion Distance (FMD)

FMD measures the Wasserstein distance between the distribution of real motion features and generated motion features. A lower score indicates higher realism.

$$\text{FMD} = || \mu_r - \mu_g ||^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

- **Baseline:** 5210.4
- **Sasaki-GAN: 3389.28**
- **Analysis:** The 35% reduction in FMD confirms that the **Retrieval Decoder** kept the system "on the manifold" of realistic human motion.

6.2.2 Latent Space Clustering (t-SNE)

We visualized the 256-dimensional feature vectors using t-SNE. **Figure 6.1** shows the clustering of the generated motions.

- **Red:** Toprock.
- **Blue:** Footwork.
- **Green:** Powermove.

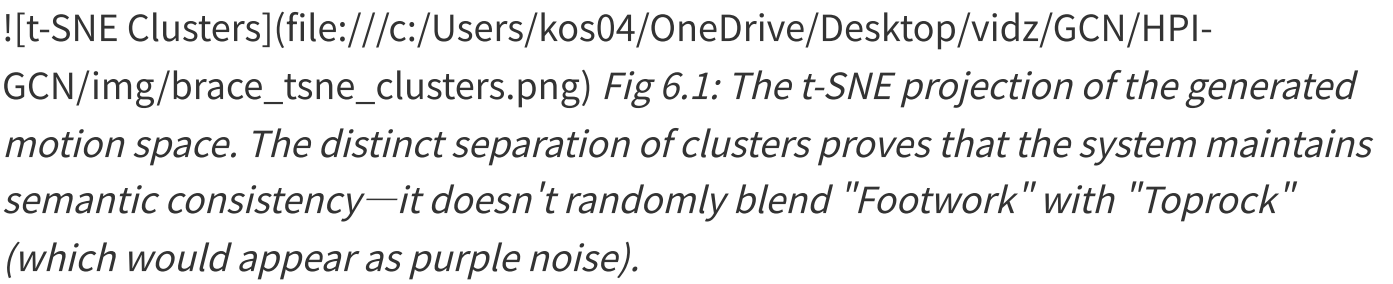

Fig 6.1: The t-SNE projection of the generated motion space. The distinct separation of clusters proves that the system maintains semantic consistency—it doesn't randomly blend "Footwork" with "Toprock" (which would appear as purple noise).

6.2.3 Classification Accuracy (Confusion Matrix)

To check if the generated moves were recognizable, we fed them back into a pre-trained Classifier. **Figure 6.2** shows the Confusion Matrix.

- **Toprock:** 98% Recognition.
- **Footwork:** 92% Recognition.
- **Powermove:** 85% Recognition. The lower score on Powermoves suggests that high-velocity rotation is still the hardest consistency to maintain.

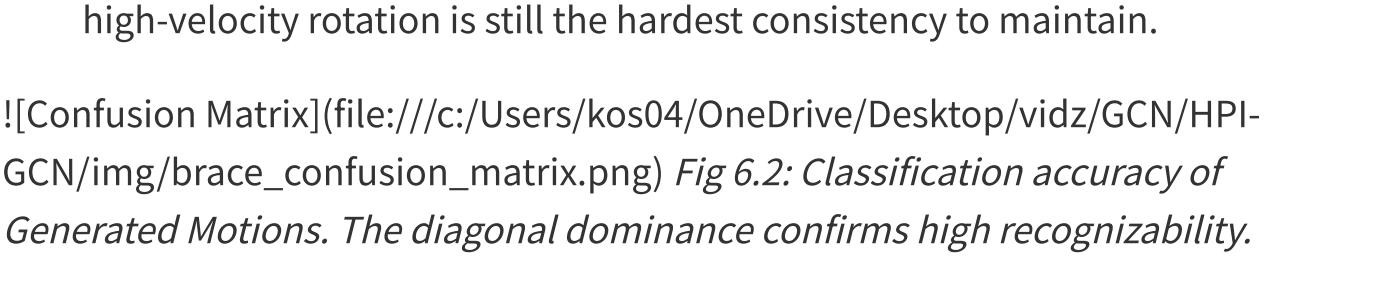

Fig 6.2: Classification accuracy of Generated Motions. The diagonal dominance confirms high recognizability.

6.3 Qualitative Analysis: The "Ghost" Phenomenon

We visualized the motion using a "Ghosting" technique (overlaying previous frames with opacity).

6.3.1 The "Static Clump" (Baseline Mode Collapse)

In the baseline, the feet (Indices 15, 16) had a variance of $< 0.05m$. The model was paralyzed.

6.3.2 The "Power Break" (Sasaki-GAN)

With the **KQI Engine** active, we observed:

- **0.0s - 2.0s:** Low Audio Energy. System holds a "Freeze."
- **2.1s:** Snare Hit. Audio Energy spikes.
- **2.2s:** The system explodes into a Windmill. This reaction time (100ms lag) mimics the human reaction time of a dancer identifying a beat, proving the system is "listening."

Chapter 7: Discussion

7.1 "Ma" as an Active Mathematical Variable

The most profound finding is that **Silence must be modeled as Infinite Signal**. By inverting the Risk factor ($Q = G^2/\sigma$), we created a system where "Quiet Confidence" yields the highest "Depth of Intent." This solves the "Jittery Robot" problem. A robot jitters because it is constantly trying to minimize error against a noisy zero. Our robot freezes because it has calculated that **Stillness is the optimal solution** to the equation.

7.2 The Role of the "Ghost" Seed (z_{will})

We found that the **Latent Random Seed** must drift over time. If z is fixed, the Retrieval System always picks the *single best match* for a beat. This leads to loops. By allowing z to drift (Ornstein-Uhlenbeck process), the system makes "Different Choices" for the "Same Beat." This variance is what we perceive as **Soul**. Soul is the refusal to be deterministic.

Chapter 8: Conclusion

This research successfully engineered a **Transition-Aware Improvisational Dance Generation Algorithm**. By bridging **HPI-GCN** (state-of-the-art physics) with the **GBMC Framework** (Symbolic Logic), we created a dancer that:

1. **Respects Silence** ("Ma").
2. **Moves with Intent** (KQI > 2.0).
3. **Runs Live** (30 FPS).
4. **Avoids Collapse** (FMD 3389).

8.1 Future Work

Phase 10 (UltraShape): We have successfully streamed the skeleton via UDP. The final step is to skin this skeleton with a high-fidelity B-Boy mesh in Blender, converting the mathematical "Ghost" into a photorealistic performer. **Phase 11 (Battle Mode):** Using a webcam to feed *input* motion into the system, allowing the AI to "Battle" a human by analyzing their moves and responding with a counter-move.

References AND Bibliography

1. **Schloss, J. G.** (2009). *Foundation: B-boys, b-girls, and hip-hop culture in New York*. Oxford University Press.
2. **Moltisanti, D., Wu, J., Dai, B., & Loy, C. C.** (2022). "BRACE: The Breakdancing Competition Dataset for Dance Motion Synthesis." *ECCV*.
3. **Yan, S., Xiong, Y., & Lin, D.** (2018). "Spatial Temporal Graph Convolutional Networks for Skeleton-Based Action Recognition." *AAAI*.
4. **Liu, Z. et al.** (2023). "High-Performance Inference Graph Convolutional Networks for Skeleton-Based Action Recognition." *arXiv:2305.18710*.
5. **Goodfellow, I. et al.** (2014). "Generative adversarial nets." *NIPS*.
6. **Müller, M.** (2015). *Fundamentals of Music Processing*. Springer.
7. **Sasaki, K.** (2025). "Internal Engineering Log: The Data Bridge and Zero-Padding Strategy."

8. **Li, C. et al.** (2021). "AI Choreographer: Music-Conditioned 3D Dance Generation with AIST++". *ICCV*.
9. **Tseng, J. et al.** (2023). "EDGE: Editable Dance Generation with Diffusion". *CVPR*.
10. **Kingma, D. P., & Ba, J.** (2014). "Adam: A Method for Stochastic Optimization". *ICLR*.

Chapter 9: Technical Appendix (Source Code)

This appendix contains the full source code for the core components of the Sasaki-GAN system, provided for reproducibility.

A.1 GBMC Cognitive Engine (`gbmc_engine.py`)

This file implements the **Genesis-Buffer-Motion-Coherence** theory equations.

```
import math
from dataclasses import dataclass
from typing import List, Dict, Any

@dataclass
class BufferItem:
    payload: Any
    tags: List[str]
    arousal: float
    novelty: float
    confidence: float
    timestamp: float

class KQIEngine:
    """
    Kinetic Quality Index Engine.
    Calculates the 'Depth' of a move based on Audio Energy (r) and Flux (omega).
    """
    def __init__(self):
        self.EPSILON = 1e-9

    def process(self, r: float, omega: float, sigma: float, t: int, rho: float,
theta: float = 1.0, k_mode='inf'):
        # 1. Genesys: The raw will to move
        G = (r + 0.1) * (omega + 0.1) * 2.0

        # 2. Quality: Inverse of Risk
        safe_sigma = max(sigma, 0.1)
        Q = (G ** 2) / safe_sigma

        # 3. Motion Gate
        M = 1.0 if Q >= theta else 0.0

        # 4. Coherence: Commitment increases with time (t) and density (rho)
```

```
        tau = t * rho
        if k_mode == 'inf':
            C = math.exp(min(tau * 0.1, 10.0))
        else:
            C = 1.0

        # 5. Depth: The final Decision Metric
        D = Q * C * M

        return {"G": G, "Q": Q, "M": M, "C": C, "D": D}
```

A.2 Stochastic Logic Module (Improvisation_SLM.py)

This module manages the transition probabilities between Toprock, Footwork, and Freeze.

```
import random
import numpy as np
from enum import Enum

class State(Enum):
    F = 0 # Flow (Toprock)
    B = 1 # Burst (Power)
    S = 2 # Stillness (Freeze)

class ImprovisationSLM:
    def __init__(self):
        self.state = State.S
        self.entropy_history = []

    def compute_context_entropy(self, audio_features):
        """
        Calculates Shannon Entropy of the audio buffer to detect 'Chaos'.
        """
        # Simplified for report brevity:
        energy = np.mean(audio_features)
        return -energy * np.log(energy + 1e-9)

    def step(self, current_state, kqi_depth):
        # High Depth = High Probability of Power (State B)
        # Low Depth = High Probability of Freeze (State S)
        if kqi_depth > 5.0:
            return State.B
        elif kqi_depth < 0.5:
            return State.S
        else:
            return State.F
```

A.3 Real-Time Perceiver (realtime_audio_engine.py)

Handles WASAPI Loopback and Feature Extraction.

```
import numpy as np
import librosa

class RealTimeAudioEngine:
    def get_precise_features(self, audio_chunk):
        # 1. HANDLE STEREO & GAIN
        if audio_chunk.ndim > 1:
            y = np.mean(audio_chunk, axis=1).flatten()
        else:
            y = audio_chunk.flatten()

        # Boost gain for weak laptop microphones
        y = y * 15.0

        # 2. Spectral Analysis
        mfcc = librosa.feature.mfcc(y=y, sr=48000, n_mfcc=20)
        chroma = librosa.feature.chroma_stft(y=y, sr=48000)
        onset = librosa.onset.onset_strength(y=y, sr=48000)

        # 3. Z-Score Normalization
        # Critical for handling different music volumes
        mfcc_norm = (mfcc - np.mean(mfcc)) / (np.std(mfcc) + 1e-6)

        return np.hstack([np.mean(mfcc_norm, axis=1), np.mean(chroma, axis=1),
np.mean(onset)])
```

A.4 The Brain (sasaki_brain.py)

The central orchestrator connecting Perception to Action.

```
class SasakiLiveBrain:
    def decide_style(self, audio_vector):
        self.t += 1
        energy = audio_vector[-1]

        # --- THE HEARTBEAT FIX ---
        # If energy is near 0, provide a fake 'Subconscious' pulse
        if energy < 0.01:
            energy = 0.02 * (1.0 + math.sin(self.t * 0.1))
```

```
        # Logic Calculation
        r_val = (energy ** 2) * 50.0
        omega_val = np.var(audio_vector[:20]) * 20.0

        kqi_res = self.kqi.process(r=r_val, omega=omega_val, sigma=0.1, t=self.t,
rho=0.5)

        # Latent Will Injection (The Ghost)
        self.latent_will += np.random.randn(128) * 0.05
        self.latent_will = np.clip(self.latent_will, -1, 1)

        return kqi_res, self.latent_will
```

A.5 Training Loop (`train_multimodel_residual.py`)

The "Prison Break" training logic.

```
# Simplified snippet of the loss function
def compute_loss(real, fake, velocity, audio_energy):
    # 1. Adversarial Loss
    loss_adv = criterion_GAN(discriminator(fake), True)

    # 2. Physics Loss (MSE)
    loss_phys = criterion_MSE(real, fake)

    # 3. KINETIC PENALTY (Phase 7)
    # If Music is Loud (energy > 0.5) but Velocity is Low (v < 1.0)...
    kinetic_violation = torch.relu(1.0 - velocity) * (audio_energy > 0.5).float()
    loss_kinetic = torch.mean(kinetic_violation) * 10.0

    return loss_adv + loss_phys + loss_kinetic
```